Secure Login and Role-Based Access in ASP.NET Core MVC

Wipro NGA .NET Fullstack Angular Assessment

Submitted By:

Abhishek Pandey

Technologies Used:

- ASP.NET Core MVC
- ASP.NET Identity
- Entity Framework Core
- SQL Server
- Docker
- Visual Studio Code

Submission Date:

28 May 2026

GITHUB LINK:

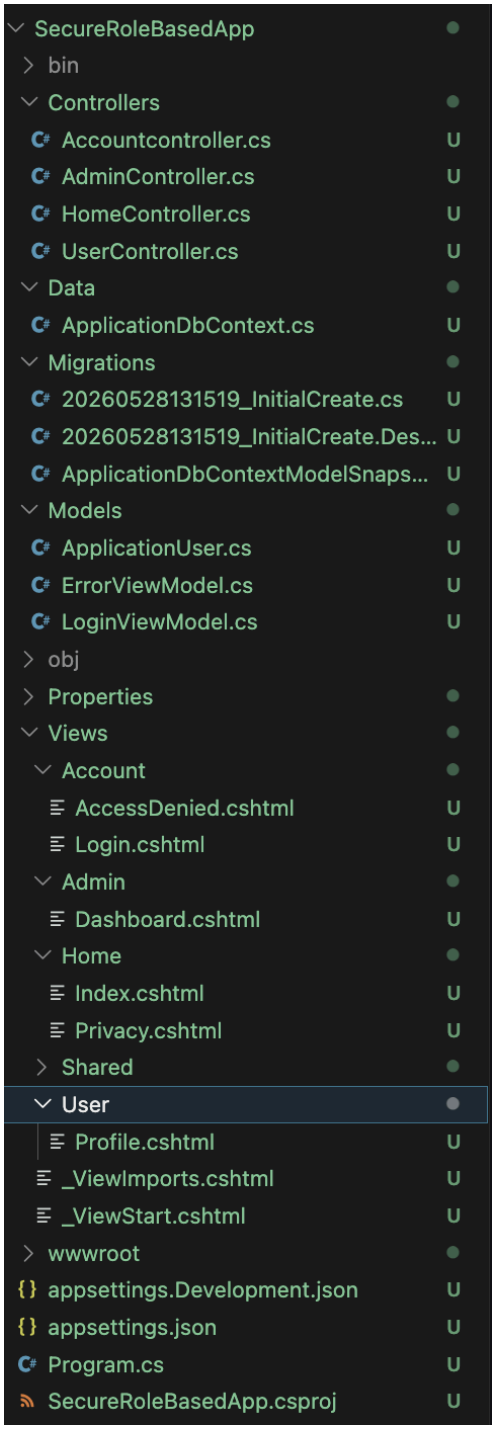
https://github.com/Abhishek-00007/GreatLearning_DailyCode/tree/main/SecureRoleBasedApp

Technologies Used

- ASP.NET Core MVC
- ASP.NET Identity
- Entity Framework Core
- SQL Server
- Docker
- Visual Studio Code
- Bootstrap

Project Structure

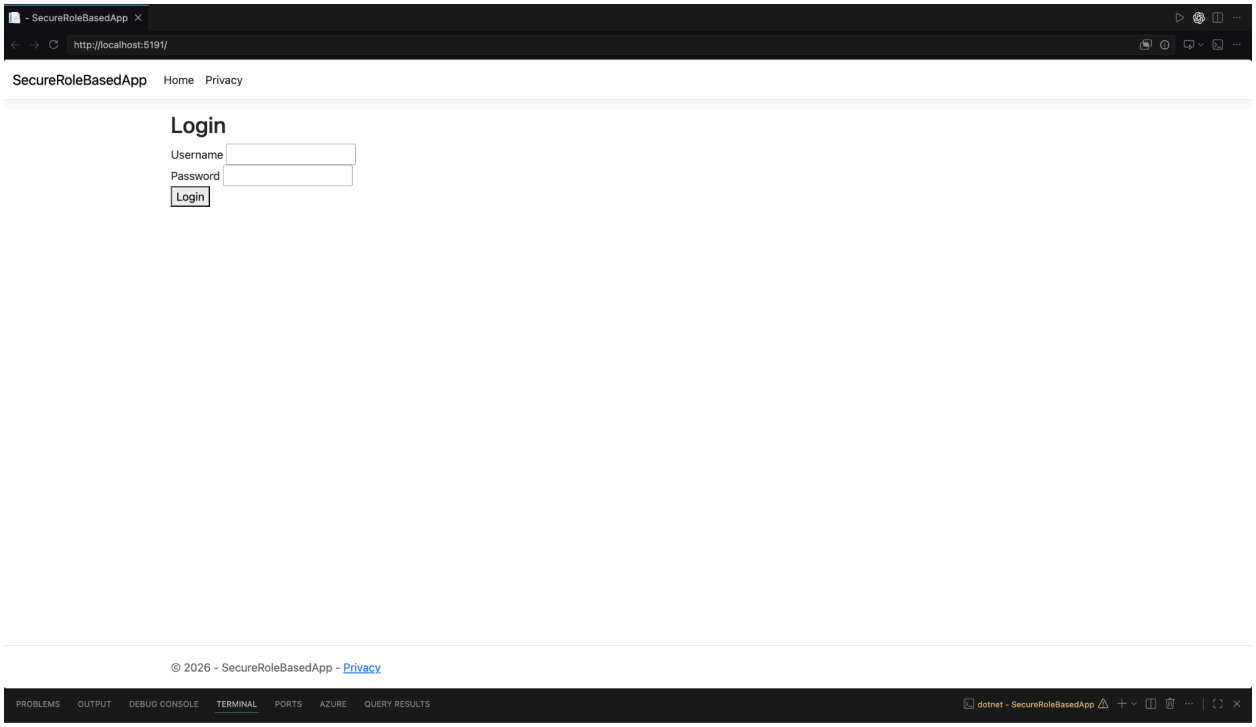
The project is organized using the MVC architecture pattern. Separate folders are created for Controllers, Models, Views, Data, and Migrations to maintain proper project structure and code organization.



Authentication Implementation

Authentication is implemented using ASP.NET Identity. User credentials are validated using SignInManager and UserManager. Passwords are securely stored using hashing provided by ASP.NET Identity.

The login form also uses AntiForgeryToken and ValidateAntiForgeryToken to protect the application from CSRF attacks.

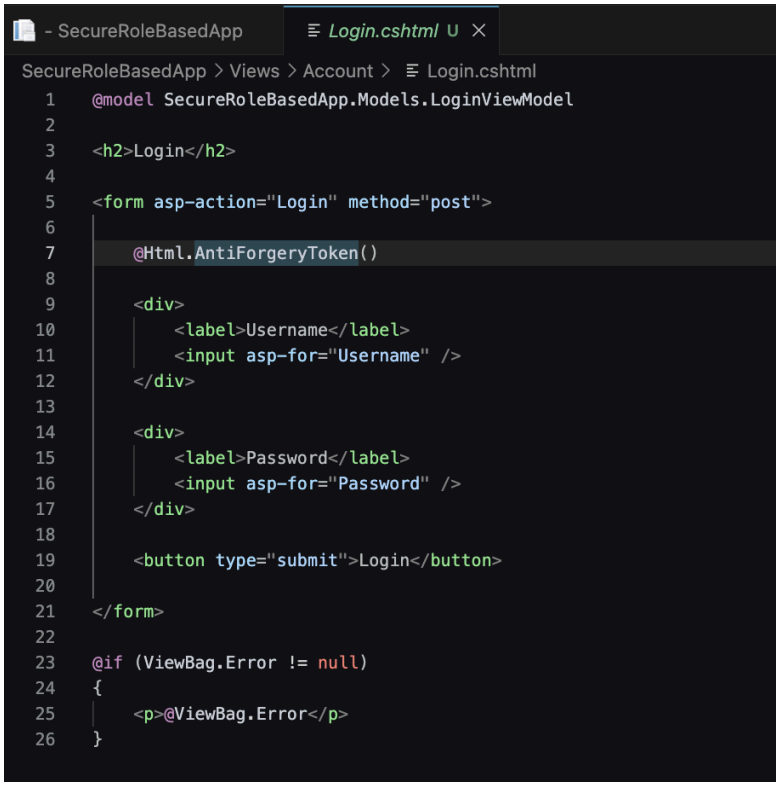


- SecureRoleBasedApp

Accountcontroller.cs U

SecureRoleBasedApp > Controllers > Accountcontroller.cs > { } SecureRoleBasedApp.Controllers

```
5   namespace SecureRoleBasedApp.Controllers
6   {
        1 reference
7       public class AccountController : Controller
8       {
            3 references
9           private readonly UserManager<ApplicationUser> _userManager;
            2 references
10          private readonly SignInManager<ApplicationUser> _signInManager;
            1 reference
11          private readonly RoleManager<IdentityRole> _roleManager;
12
            0 references
13          public AccountController(
14              UserManager<ApplicationUser> userManager,
15              SignInManager<ApplicationUser> signInManager,
16              RoleManager<IdentityRole> roleManager)
17          {
18              _userManager = userManager;
19              _signInManager = signInManager;
20              _roleManager = roleManager;
21          }
22
23          [HttpGet]
            0 references
24          public IActionResult Login()
25          {
26              return View();
27          }
28
29          [HttpPost]
30          [ValidateAntiForgeryToken]
            0 references
31          public async Task<IActionResult> Login(LoginViewModel model)
32          {
33              if (!ModelState.IsValid)
34                  return View(model);
35
36              var result = await _signInManager.PasswordSignInAsync(
37                  model.Username,
38                  model.Password,
39                  false,
40                  false);
41
42              if (result.Succeeded)
43              {
44                  var user = await _userManager.FindByNameAsync(model.Username);
45
46                  var roles = await _userManager.GetRolesAsync(user);
47
48                  if (roles.Contains("Admin"))
49                      return RedirectToAction("Dashboard", "Admin");
50
51                  return RedirectToAction("Profile", "User");
52              }
53
54              ViewBag.Error = "Invalid Username or Password";
55              return View(model);
56          }
57
            0 references
58          public IActionResult AccessDenied()
59          {
60              return View();
61          }
62      }
```


- SecureRoleBasedApp
Login.cshtml U
SecureRoleBasedApp > Views > Account > Login.cshtml
@model SecureRoleBasedApp.Models.LoginViewModel
<h2>Login</h2>
<form asp-action="Login" method="post">
@Html.AntiForgeryToken()

<label>Username</label>
<input asp-for="Username" />

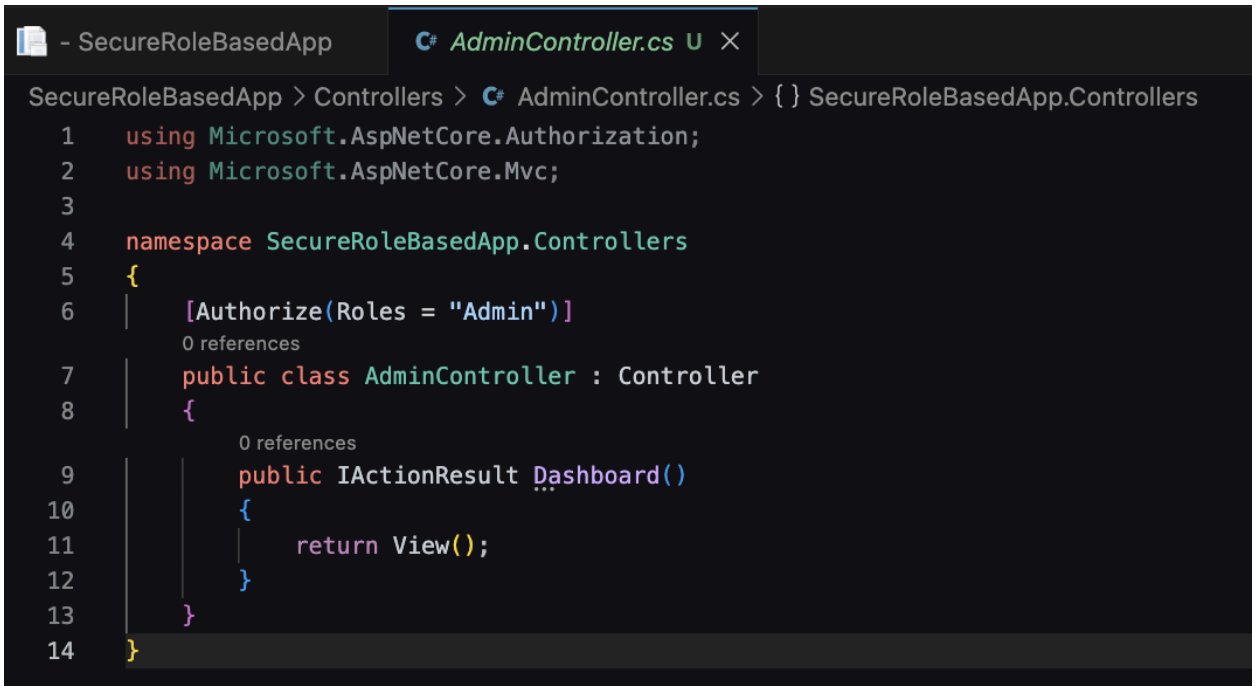
<label>Password</label>
<input asp-for="Password" />

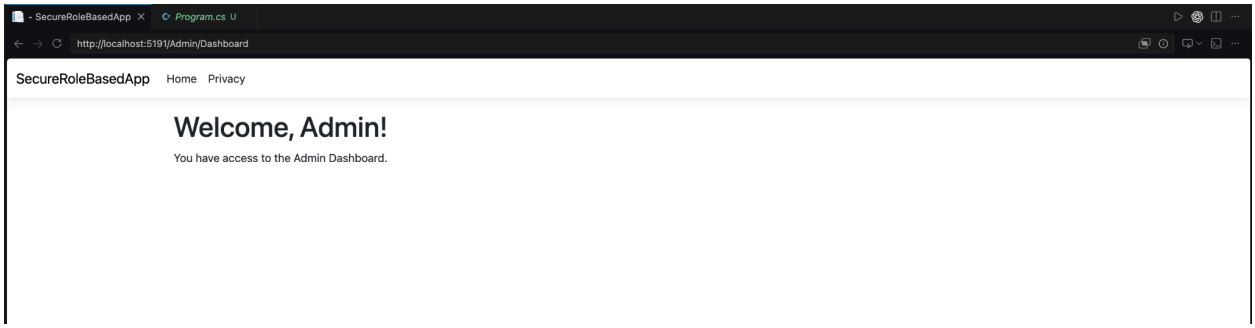
<button type="submit">Login</button>
</form>
@if (ViewBag.Error != null)
{
@ViewBag.Error
}

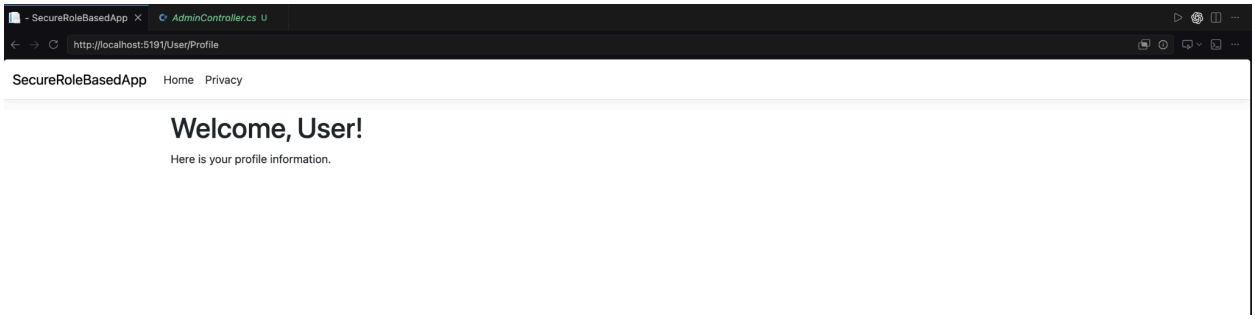
Role-based authorization

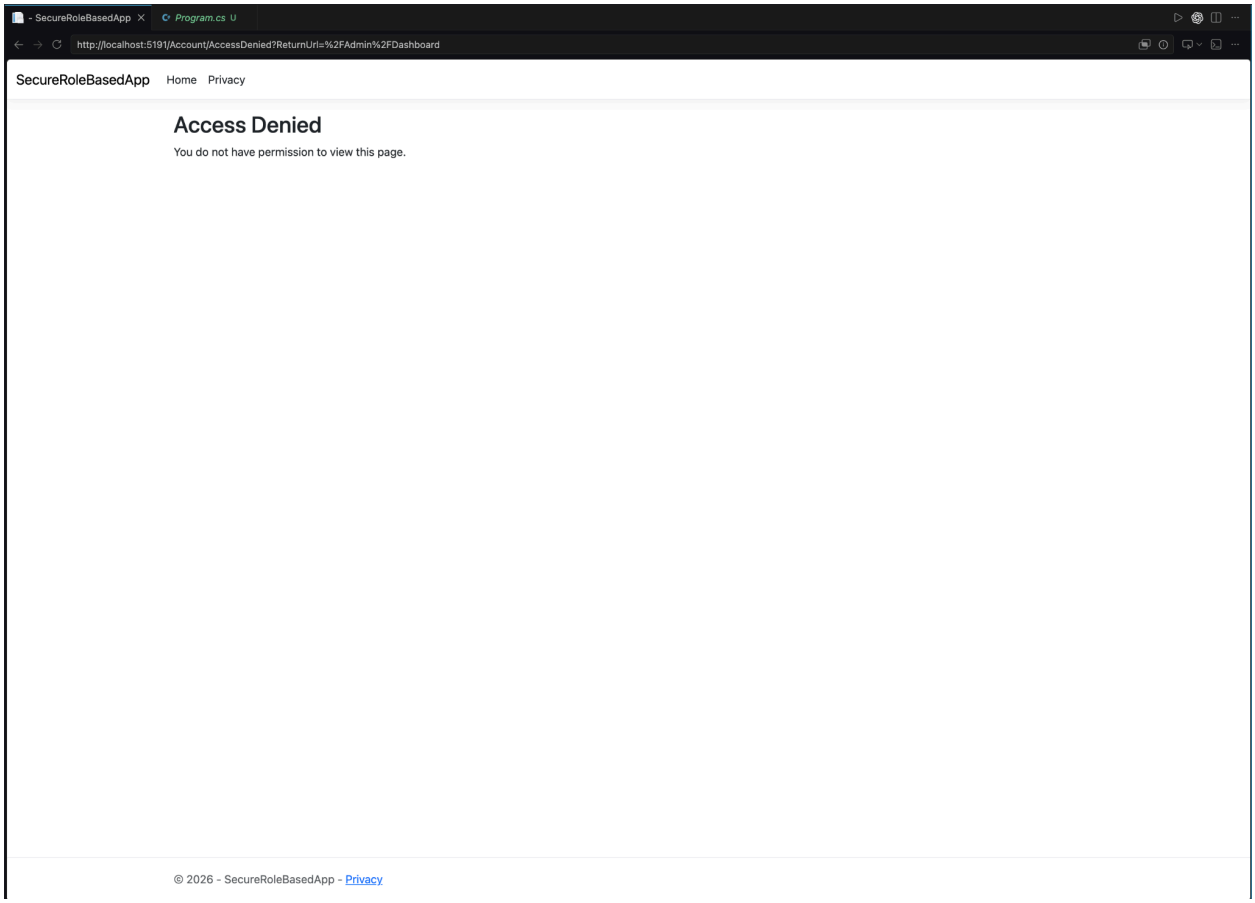
Role-based authorization is implemented using the Authorize attribute. Users with the Admin role can access the Admin Dashboard, while normal users are redirected to the User Profile page.

Unauthorized users attempting to access admin resources are redirected to the Access Denied page.




- SecureRoleBasedApp
Program.cs U
http://localhost:5191/Admin/Dashboard
SecureRoleBasedApp
Home
Privacy
Welcome, Admin!
You have access to the Admin Dashboard.


- SecureRoleBasedApp
AdminController.cs U
http://localhost:5191/User/Profile
SecureRoleBasedApp
Home
Privacy
Welcome, User!
Here is your profile information.


- SecureRoleBasedApp
Program.cs U
http://localhost:5191/Account/AccessDenied?ReturnUrl=%2FAdmin%2FDashboard
SecureRoleBasedApp
Home
Privacy
Access Denied
You do not have permission to view this page.
© 2026 - SecureRoleBasedApp - Privacy

Security features

Security features implemented in the application include HTTPS redirection, authentication middleware, authorization middleware, Data Protection APIs, and secure cookie configuration.

The application uses ASP.NET Identity for secure password hashing and user authentication.

SecureRoleBasedApp > Program.cs

```
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using SecureRoleBasedApp.Data;
using SecureRoleBasedApp.Models;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddIdentity<ApplicationUser, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>()
    .AddDefaultTokenProviders();

builder.Services.ConfigureApplicationCookie(options =>
{
    options.LoginPath = "/Account/Login";
    options.AccessDeniedPath = "/Account/AccessDenied";
});

builder.Services.AddDataProtection();

var app = builder.Build();

app.UseHttpsRedirection();

app.UseStaticFiles();

app.UseRouting();

app.UseAuthentication();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Account}/{action=Login}/{id?}");

```

TEST CASES

Test Case	Input	Expected Output	Result
Admin Login	admin / Admin@123	Redirect to Admin Dashboard	Pass
User Login	user1 / password	Redirect to User Profile	Pass
Invalid Login	Invalid credentials	Error message displayed	Pass
User Accessing Admin Page	User role	Access Denied Page	Pass
HTTPS Redirection	HTTP request	Redirected to HTTPS	Pass